Django Expert Advice Platform Structure

https://gemini.google.com/app/6e59bdc5d5582730

User prompt: i have to create a complete django project advise app where there are two types of users experts and users the expert should asked for neccessary information like catogory and subcatogary which is stored in database and also asked for location with state and dist , city and for the user just get the basic info now when you login as user the user should see the bio of expert and he can search the expert by name, catagory , subcatagory and location and now when the user is confirmed the expert now he can send a request to expert now the expert see the request and he can accept , reject and when he accept the user will get the notification to make the payment and this payment is valid for one day and when the request is accepted the experts schedule will be fixed and other users will see as occupied so give me structure to make this project

Response: Building an Expert Advice platform in Django requires a clean separation of roles, a robust state machine for request handling, and clear search/filtering capabilities.

Here is a complete architectural blueprint and implementation structure for your project.

1. Project Architecture & Directory Structure

It's best to split this project into three core apps:

- **`accounts`**: Custom user model, profiles (Expert vs. User), and authentication.
- **`advice`**: Categories, locations, search views, and expert directory.
- **`consultations`**: Requests, schedule locks, payment integration, and status transitions.

```
advice_platform/
├── manage.py
├── advice_platform/           # Project configuration
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── apps/
│   ├── accounts/              # Users & Profiles
│   │   ├── models.py          # CustomUser, ExpertProfile, UserProfile
│   │   ├── views.py           # Registration, Profile setup
│   │   └── urls.py
│   ├── advice/                # Core Catalog & Search
│   │   ├── models.py          # Category, SubCategory, State, District, City
│   │   ├── views.py           # Expert Search & Directory
│   │   └── filters.py         # django-filter implementation
│   └── consultations/         # Booking & Payments
│       ├── models.py          # AdviceRequest, Schedule, Payment
│       ├── views.py           # Request handling, Accept/Reject, Payment
│       └── tasks.py           # Celery/Cron task for 1-day payment expiry
```

2. Core Data Models Design

`accounts/models.py`

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    class Role(models.TextChoices):
        EXPERT = "EXPERT", "Expert"
        USER = "USER", "User"

    role = models.CharField(max_length=10, choices=Role.choices, default=Role.USER)

class ExpertProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name="expert_profile")
    category = models.ForeignKey("advice.Category", on_delete=models.SET_NULL, null=True)
    subcategory = models.ForeignKey("advice.SubCategory", on_delete=models.SET_NULL, null=True)

    # Location details
    state = models.ForeignKey("advice.State", on_delete=models.SET_NULL, null=True)
    district = models.ForeignKey("advice.District", on_delete=models.SET_NULL, null=True)
    city = models.CharField(max_length=100)

    bio = models.TextField()
    is_occupied = models.BooleanField(default=False)  # Quick availability flag

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name="user_profile")
    phone = models.CharField(max_length=15, blank=True)
```

`advice/models.py`

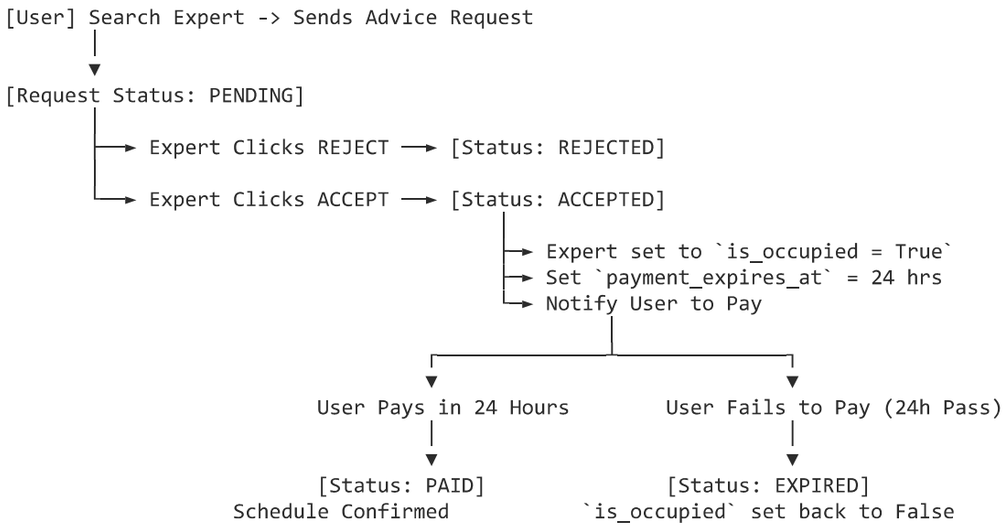
```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=100, unique=True)

class SubCategory(models.Model):
    category = models.ForeignKey(Category, on_delete=models.CASCADE, related_name="subcategories")
```

```
    name = models.CharField(max_length=100)

class State(models.Model):
    name = models.CharField(max_length=100)

class District(models.Model):
    state = models.ForeignKey(State, on_delete=models.CASCADE, related_name="districts")
    name = models.CharField(max_length=100)
```

consultations/models.py

```
from django.db import models
from django.utils import timezone
import datetime

class AdviceRequest(models.Model):
    class Status(models.TextChoices):
        PENDING = "PENDING", "Pending"
        ACCEPTED = "ACCEPTED", "Accepted (Payment Pending)"
        REJECTED = "REJECTED", "Rejected"
        PAID = "PAID", "Confirmed & Paid"
        EXPIRED = "EXPIRED", "Payment Window Expired"

    user = models.ForeignKey("accounts.User", on_delete=models.CASCADE, related_name="sent_requests")
    expert = models.ForeignKey("accounts.User", on_delete=models.CASCADE, related_name="received_requests")
    requested_slot = models.DateTimeField()
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.PENDING)

    payment_expires_at = models.DateTimeField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def set_accepted(self):
        """When Expert accepts, trigger payment deadline and lock schedule."""
        self.status = self.Status.ACCEPTED
        self.payment_expires_at = timezone.now() + datetime.timedelta(days=1)
        self.save()

        # Mark expert as occupied
        self.expert.expert_profile.is_occupied = True
        self.expert.expert_profile.save()
```

3. Workflow & Request Lifecycle



4. Implementation Step-by-Step

1 **Configure Custom User & Models**

1. Define `User` with `role` field in `accounts/models.py`.
2. Set `AUTH_USER_MODEL = 'accounts.User'` in `settings.py` before running initial migrations.
3. Create `Category`, `SubCategory`, `State`, and `District` models and register them in Django Admin.

2 **Build Search & Filtering Views**

Use `django-filter` to let users filter experts by name, category, subcategory, state, district, and city:

```
import django_filters
from accounts.models import ExpertProfile

class ExpertFilter(django_filters.FilterSet):
    name = django_filters.CharFilter(field_name="user__first_name", lookup_expr="icontains")

    class Meta:
```

```
        model = ExpertProfile
        fields = ['category', 'subcategory', 'state', 'district', 'city']
```

3 **Implement Booking & Acceptance Logic**

Write views for the expert dashboard where they can accept/reject pending requests. Update the status and trigger the 24-hour payment timer on acceptance.

4 **Set Up Payment Expiry Management**

Use **Celery** with **Celery Beat** or a simple scheduled management command running via Cron to release occupied slots:

```
# Periodic check for unpaid requests
expired_requests = AdviceRequest.objects.filter(
    status=AdviceRequest.Status.ACCEPTED,
    payment_expires_at__lt=timezone.now()
)
for req in expired_requests:
    req.status = AdviceRequest.Status.EXPIRED
    req.save()
    req.expert.expert_profile.is_occupied = False
    req.expert.expert_profile.save()
```
